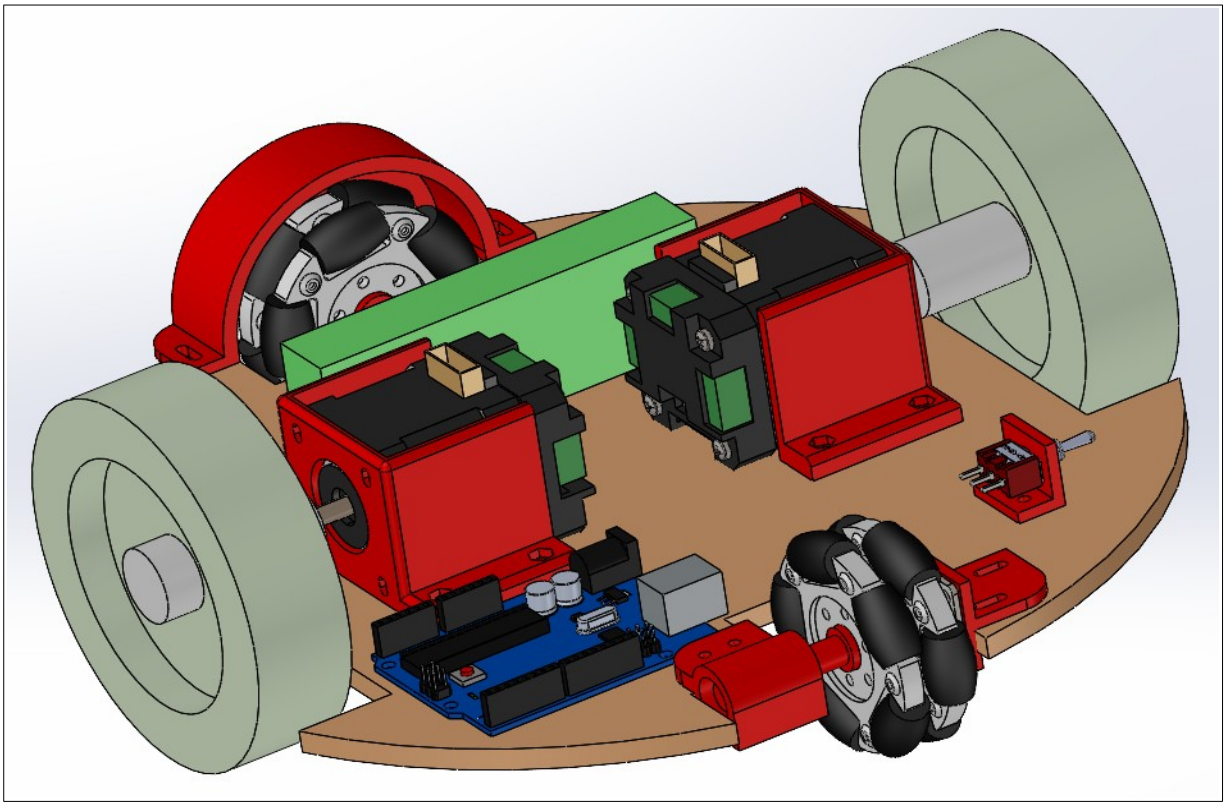
Rapport de projet

L1 - CMI

Session 2026

Challenge Robot déplacement précis



Centre d'Enseignement et de Recherche en Informatique
Campus Jean Henri Fabre
339 Chemin des Meinajaries
84911 Avignon Cedex 9

Auteurs :

Ethan Bernon

Noah Barbié Spina

Kelian Raoux

Encadrants :

M. Zouania

M. Silanus

Résumé

L’objectif de ce projet était de réaliser un robot qui effectue des déplacements précis. Pour cela, nous avons choisi d’utiliser 2 roues classiques motorisées par des SERVO 42D 485, ainsi que 2 roues holonomes avec des encodeurs magnétiques avec capteurs d’angles à effet hall afin de mesurer la distance parcourue avec la première roue libre holonome, qui sera placée à l’avant du robot, parallèlement aux roues motrices, et la deuxième sera placée à l’arrière du robot, perpendiculairement aux roues motrices, de manière à mesurer les rotations pour savoir quel est la distance exacte parcourue et combien il nous reste à parcourir.

Pour envoyer des instructions aux moteurs avec la carte ESP32, nous avons créé une application qui peut se connecter à l’ESP32 en Bluetooth et lui envoyer des instructions de déplacement comme avancer, reculer, tourner à droite ou à gauche, en lui donnant un entier, interprété comme une distance en millimètre pour les instructions avancer ou reculer, et interprété comme des degrés pour les instructions de tourner à droite ou à gauche.

Au moment de la rédaction de ce rapport nous avons pu compléter certains de nos objectifs, mais d’autres sont en cours de réalisation. Une grande partie sur la communication entre les capteurs à effet Hall et les moteurs reste à faire ainsi que les tests finaux sur la réussite du robot pour remplir son objectif de précision.

Table des matières

1 Introduction....4
1.1 Contexte....4
1.2 Problématique....4
2 État de l'art....4
2.1 Solution 1....5
2.2 Solution 2....5
2.3 Solution 3....5
2.4 Solution 4....6
2.5 Solution 5....6
3 Solution développée....6
4 Organisation....6
4.1 L'équipe de projet....6
4.2 Répartition des tâches....7
4.3 Environnement matériel/logiciel....7
4.4 Planification....8
5 Réalisation....8
5.1 Travail d'équipe....8
5.2 Partie d'Ethan Bernon....9
5.3 Partie de Kelian Raoux....11
5.4 Partie de Noah Barbié Spina....15
5.5 Intégration....21
6 Conclusion....21

Robot déplacement précis

1 Introduction

1.1 Contexte

Ce projet est inscrit dans le contexte de l'UE CMI 3 : interfaces du réel au numérique (S-U09-4103) du semestre 2 de la licence informatique à l'université d'Avignon.

1.2 Problématique

Pour ce projet, le cahier des charges était clair : Gérer les déplacements élémentaires du robot via les points suivants :

- Une partie matérielle : carte Arduino, cartes de commande MD25 avec moteurs et encodeurs, moteurs pas à pas et roues omnidirectionnelles, base roulante (existante ou à créer), support capteurs à effet Hall, conception/impression 3D.
- Une partie logicielle : Application de gestion des déplacements du robot (protocole de communication r2t2)
- Ingénierie : Imaginer, mettre en œuvre des solutions et corriger des problèmes.
- Innovation : Déplacement autonome d'un robot ou d'un véhicule avec la plus grande précision.

2 État de l'art

La robotique mobile de précision se décline en trois familles : la logistique automatisée, la robotique de compétition et les plateformes pédagogiques. Ce projet s'inscrit dans la troisième catégorie.

La première catégorie regroupe des robots principalement utilisés dans le contexte industriel. On y retrouve les AGV (Automated Guided Vehicle) et les AMR (Autonomous Mobile Robot). Bien qu'ils soient tous deux utilisés pour le transport de matériaux, il existe une différence fondamentale dans leur fonctionnement et leur flexibilité.

- Les AGV sont des robots qui suivent des chemins fixes ou des pistes pour le transport de matériel, nécessitant généralement des changements d'infrastructure comme des bandes magnétiques ou des fils.
- Les AMR sont des robots qui fonctionnent de manière autonome et qui peuvent naviguer dans un environnement non contrôlé sans avoir besoin de chemins ou de pistes fixes. Ils sont connus pour leur flexibilité et constituent une partie importante de toutes les solutions nécessitant de la productivité dans les opérations et la logique de fabrication.

La deuxième famille est celle de la robotique de compétition, illustrée par des événements comme Eurobot ou la RoboCup, où des robots autonomes évoluent sur des terrains balisés avec des contraintes de précision millimétriques, de rapidité d'exécution et de robustesse face à l'adversaire. Ces plateformes constituent une référence technique exigeante, souvent à l'origine d'innovations en asservissement et en localisation.

La troisième famille, celle des plateformes pédagogiques et de prototypage, regroupe les systèmes conçus dans un cadre éducatif ou de recherche appliquée, où la priorité est donnée à la compréhension des mécanismes, à la modularité et à l'accessibilité des composants.

2.1 Solution 1

L’asservissement est la maîtrise du déplacement de précision reposant en premier lieu sur un gestion rigoureuse des moteurs. Le correcteur PID (Proportionnel–Intégral–Dérivé), implémenté en version discrète sur microcontrôleur, constitue la solution de référence pour réguler la vitesse et la position de chaque moteur. Il agit en boucle fermée en comparant en permanence la consigne souhaitée à la mesure réelle fournie par les encodeurs, et corrige l'écart en ajustant la commande moteur. Chaque roue reçoit une consigne individuelle calculée à partir de la vitesse globale désirée du robot (translation et rotation). Au-delà du simple suivi de vitesse, cela permet de limiter les à-coups mécaniques et d’améliorer la précision du positionnement final.

2.2 Solution 2

Les roues omnidirectionnelles, aussi appelées roues suédoises ou roues Mecanum, constituent une solution mécanique permettant au robot de se déplacer dans n'importe quelle direction sans modifier son orientation. Leur principe repose sur des mini roues inclinées à 45° par rapport à l'axe de la roue principale. En combinant quatre de ces roues disposées aux coins du robot et en contrôlant indépendamment la vitesse et le sens de rotation de chacune, il est possible d'obtenir un déplacement longitudinal, transversal ou en rotation pure, ou toute combinaison de ces mouvements. Cette architecture est largement utilisée dans les AGV industriels et certains robots de compétition pour sa simplicité de mise en œuvre mécanique. Sa principale limitation réside dans le glissement des galets sur le sol, source d'erreurs odométriques, et dans la perte d'effort moteur utile due à la décomposition des forces, qui réduit l'efficacité énergétique par rapport à une roue standard.

2.3 Solution 3

Des roues holonomes sur un robot à 3 roues. Cette configuration à trois roues holonomes disposées à 120° les unes des autres est une architecture particulièrement répandue en robotique de compétition, notamment à Eurobot. Chaque roue est orientée tangentiellement au cercle inscrit dans le châssis du robot, avec ses mini roues orientées perpendiculairement à l'axe moteur. En pratique, cette configuration est plus sensible aux défauts de fabrication et de montage que la configuration Mecanum à quatre roues, car tout écart angulaire d'une roue par rapport à sa position théorique introduit une dérive systématique dans le déplacement.

2.4 Solution 4

L'odométrie est la méthode fondamentale de localisation relative d'un robot mobile : elle consiste à estimer la position et l'orientation du robot dans son repère de départ en intégrant les déplacements élémentaires mesurés par les encodeurs de roues. La précision de l'odométrie dépend directement de la résolution des encodeurs, du diamètre exact des roues et de la rigueur du modèle géométrique utilisé. Sa principale limitation est la dérive accumulative, ou dead reckoning drift : les erreurs de mesure, aussi faibles soient-elles, s'intègrent au fil du temps et conduisent à un écart croissant entre la position estimée et la position réelle. Pour limiter cette dérive, les capteurs à effet Hall haute résolution constituent une alternative avantageuse aux encodeurs optiques classiques, en offrant une mesure angulaire sans contact et robuste aux perturbations mécaniques. Dans des projets comparables, la fusion de l'odométrie avec d'autres sources de mesure comme des LiDAR est souvent mise en œuvre pour corriger périodiquement la dérive et maintenir une précision acceptable sur de longues distances.

2.5 Solution 5

Les moteurs pas à pas permettent une grande précision dans les instructions de rotation qu'on lui donne. Il tourne d'un angle fixe à chaque impulsion électrique, ce qui lui permet de se positionner précisément sans capteur de retour. Son avantage principal est donc la simplicité de commande en boucle ouverte. Dans l'industrie, les moteurs pas à pas sont omniprésents dans tous les systèmes nécessitant un positionnement répétable à faible coût. On les retrouve dans les imprimantes 2D et 3D pour le pilotage des axes d'impression, dans les têtes de lecture de disques durs, ou encore dans les automates de laboratoire.

3 Solution développée

Avec ces informations, nous avons décidé de réaliser un robot circulaire avec 4 roues dont 2 motorisées, ainsi que 2 roues libres holonomes munis d'encodeurs magnétiques avec capteurs d'angle à effet hall afin de mesurer la distance parcourue avec précision. L'un sera dans le sens de la marche pour mesurer les translations, l'autre lui sera perpendiculaire pour mesurer les rotations parcourues. Nous ferons donc de l'odométrie en essayant de limiter au maximum les accumulations d'erreurs lors du mouvement. Une carte ESP32 traitera l'information et enverra une instruction de déplacement aux moteurs selon le delta distance à parcourir restant. Les moteurs seront des moteurs pas à pas SERVO 42D 485 afin d'obtenir une grande précision de mouvement.

4 Organisation

4.1 L'équipe de projet

Notre équipe est constituée de 3 étudiants de L1 informatique CMI. Nous avons aussi été aidés par nos enseignants M. Zouania et M. Silanus en cas de besoin et pour le matériel.

Afin de communiquer entre membres du projet, nous avons utilisé l'application discord pour envoyer nos fichiers et débattre du projet mais aussi le service de Gestion de Projet que propose l'Université d'Avignon. Le compte rendu du projet ainsi que le diaporama ont été rédigés collaborativement sur Google Doc et Google Slide. Pour le rendu final du projet, nous avons créé un dépôt github avec tous nos fichiers.

4.2 Répartition des tâches

Voici le découpage individuel des tâches à réaliser :

Ethan Bernon	Noah Barbié Spina	Kelian Raoux
• Modélisation 3D • Recherches capteurs Hall • Calcul des formules mathématiques • Montage robot • Rédaction du rapport	• Programmation de ESP32 • Gestion des moteurs • Branchements électroniques • Montage robot • Rédaction du rapport	• Recherches communication Bluetooth • Création application • Montage robot • Rédaction du rapport

4.3 Environnement matériel/logiciel

Modélisation 3D : SolidWorks
Nous savions déjà comment utiliser le logiciel, qui était adapté à nos besoins et qui possédait une licence gratuite étudiante.

Modélisation des formules mathématiques : Desmos
Ce site web permet de visualiser simplement des formules et de faire des animations rudimentaires dans le besoin.

Programmation ESP32 : Arduino IDE
Nous avons déjà utilisé cet IDE qui permet une gestion des library, implémente un Serial Monitor et possède un Board Manager avec un grand choix de cartes.

Test et observation : MKS ServoD Control
L'application officielle pour contrôler les moteurs et tester une grande partie de leurs fonctionnalités. Elle intègre une interface simple et un terminal permettant de d'observer toutes les trames reçues et envoyées.

Création application : MIT App Inventor
Ce site permet de créer facilement une application en utilisant un système de codage par bloc ainsi qu'un système de "drag and drop" pour créer le visuel de l'application, beaucoup plus simple à prendre en main pour des débutants dans le milieu de création d'application.

Rédaction du rapport : Google Doc
La fonctionnalité d'écriture en simultané nous est utile lors de la rédaction collaborative de ce rapport.

Création du diaporama : Google Slide
La fonctionnalité d’écriture en simultané nous est utile lors de la rédaction collaborative du diaporama.

4.4 Planification

Ce projet a débuté le Mardi 27 Janvier 2026 et s’est déroulé jusqu’au Lundi 13 Avril 2026. Ce qui nous a laissé 76 jours de délai pour réaliser le projet. Durant ce laps de temps, il y a eu 8 séances de TP.

5 Réalisation

5.1 Travail d’équipe

Voici le rendu final de l’application. On peut se connecter à la carte ESP32 via bluetooth à l’aide du bouton situé en haut de l’écran, et le texte affiché à côté du bouton est codé de telle sorte à ce que quand l’appareil sur lequel est utilisé l’application est connecté en bluetooth, il passe de “Disconnected” à “Connected”. L’application à d’ailleurs était faite visuellement entièrement en anglais pour qu’elle puisse être utilisée facilement par toutes et tous.

Pour envoyer des instructions aux moteurs, il faut remplir la boîte de texte présente au milieu de l’écran avec uniquement des chiffres et éventuellement un caractère “-” en début d’instruction pour signaler qu’on veut reculer au lieu d’avancer par exemple. Nous avons choisi d’utiliser une file d’instructions, stockées dans le programme, afin de pouvoir envoyer toutes les instructions dont nous avons besoin au programme, avant de l’autoriser à défiler et exécuter les instructions. Pour éviter d’avoir une boîte de texte pour les instructions “avancer/reculer” et une autre pour “tourner à droite “ou “tourner à gauche”, nous avons utilisé un switch, qui selon son état permettra d'interpréter les instructions envoyées comme des distances en millimètres ou des rotations en degrés. Ce switch est situé au-dessus de la boîte de texte pour que l’utilisateur en prenne bien connaissance avant d’envoyer des instructions.

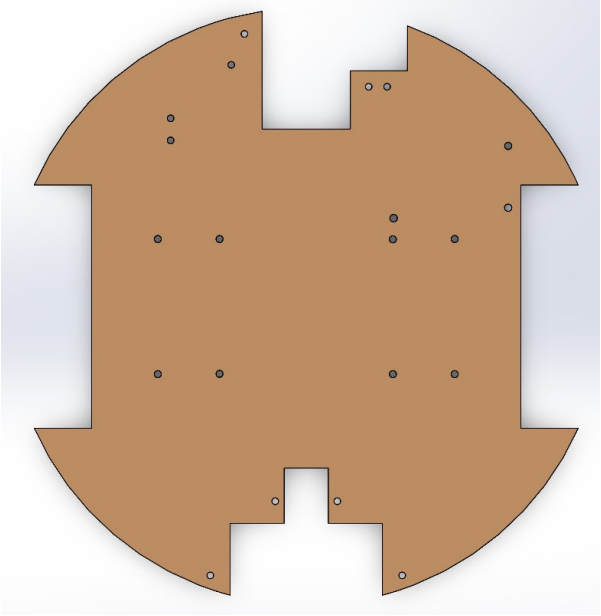
Le deuxième switch, en rouge sur l’image ci-dessus, permet quant à lui de gérer si le programme exécute les instructions ou les accumule pour les exécuter plus tard. Quand le switch est en OFF, c'est-à-dire du côté rouge, le programme n'exécute pas les instructions et les stocke dans la file. Quand on passe le switch en ON, la tête du switch devient verte, c’est à dire en mode exécution, suivant le code couleur du texte juste au dessus de lui dans l’application, ce qui envoie un message au programme pour lui dire qu’il peut commencer à défiler et exécuter les instructions. Si une instruction était en cours et que le switch passe en OFF, l’instruction en cours est arrêtée, et le programme finira de l’exécuter quand le switch repassera en ON.

5.2 Partie d’Ethan Bernon

La modélisation 3D est séparée en 2 parties : la récupération de chaque pièces individuellement sur internet ou leur création, puis l’assemblage avec toutes les contraintes liées.

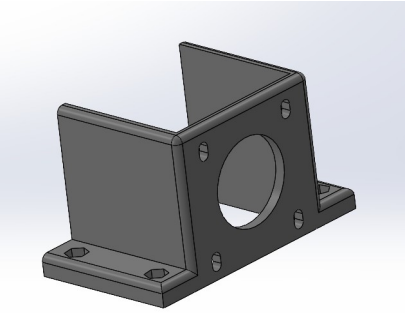
Pour les composants déjà existants soit les roues, les moteurs, les capteurs, la batterie, la carte ESP32 et l’interrupteur, nous avons cherché des modèles préexistants sur internet avec le site GrabCAD. Cette étape permet d'obtenir des mesures plus précises qu’avec nos outils car certains de ces modèles ont directement été créés par leur entreprise de production.

Néanmoins, il faut créer des pièces de support pour ces composants. La plus importante est le châssis qui est la plaque de support de chacun des composants du robot. Pour sa confection nous avons choisis une forme circulaire de taille arbitraire au début puis nous l’avons agrandie pour correspondre aux mesures des composants et pour qu’ils rentrent tous dessus. Nous y avons prépercé la plupart des trous de vis. Voici ci-contre le modèle final où l’on peut voir à droite et à gauche les emplacements pour les roues motorisées ainsi que les emplacements pour les roues libres en haut et en bas. Pour arriver à ce modèle final, sommes passés par plusieurs modèles intermédiaires car certaines de nos mesures n'étaient pas correctes ou à cause de problèmes de conceptions remarqués lors d’un assemblage test.



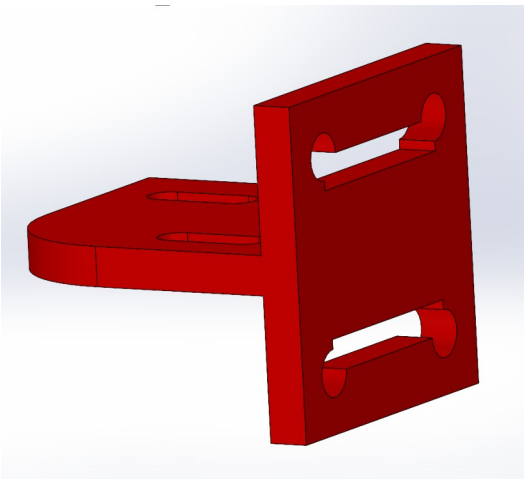
Après le châssis, nous avons fait les autres pièces de support :

- Le support des moteurs a été pensé pour englober le moteur sur les côtés afin de le stabiliser lors des rotations de la roue et il est fixé au moteur et au châssis via des vis. Il ne fait pas toute la longueur du moteur pour laisser de la place pour les câbles de branchements.
- Le support de l’interrupteur est une pièce toute simple en forme de "L" permettant à l’interrupteur d’être soutenu. Il est fixé au châssis via des vis.

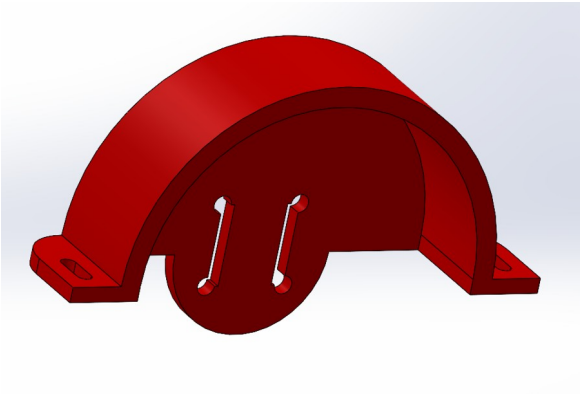


- Le support du capteur hall avant est une pièce permettant au capteur hall d’être orienté vers la roue à la bonne distance via les congés faits au niveau des trous de vis. Cette pièce doit aussi avoir un pattern spécial afin que le capteur hall se fixe à la pièce tout en laissant passer les câbles de branchement.

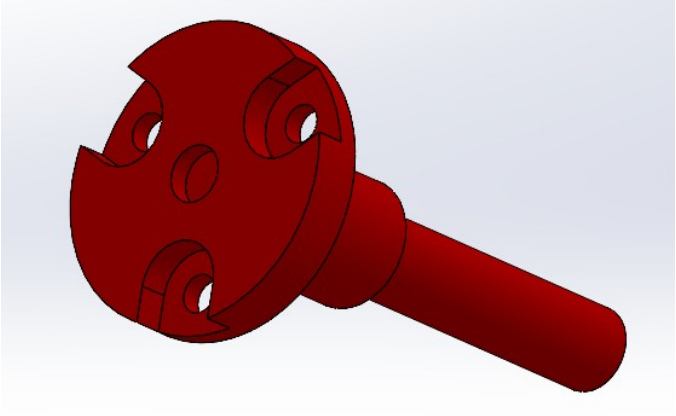
- Le support du capteur hall arrière possède les mêmes caractéristiques que le support avant mais le sens de la roue est différent (perpendiculaire au sens du robot) et cela nécessite de changer la forme globale de la pièce.



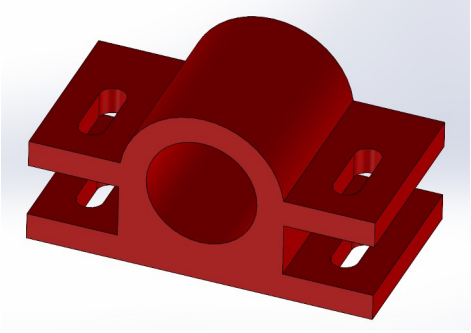
Support avant final



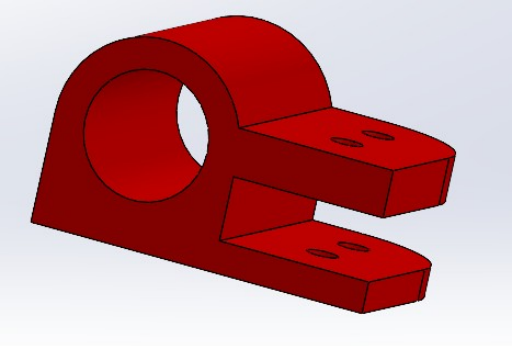
Support arrière final



- L'axe de rotation des roues libres a aussi été modélisé pour permettre d'avoir un axe fixé à la roue. Nous avons aussi laissé un espace pour insérer l'aimant qui permettra de capter les rotations de la roue.
- Le support des roulements à bille avant doit permettre de supporter la rotation des roues libres. Nous avons fait un long tube où 2 roulements à billes sont logés pour porter l'axe de rotation des roues libres. Le support doit aussi être fixé au châssis via des vis.
- Le support des roulements à billes arrière est similaire au support avant.

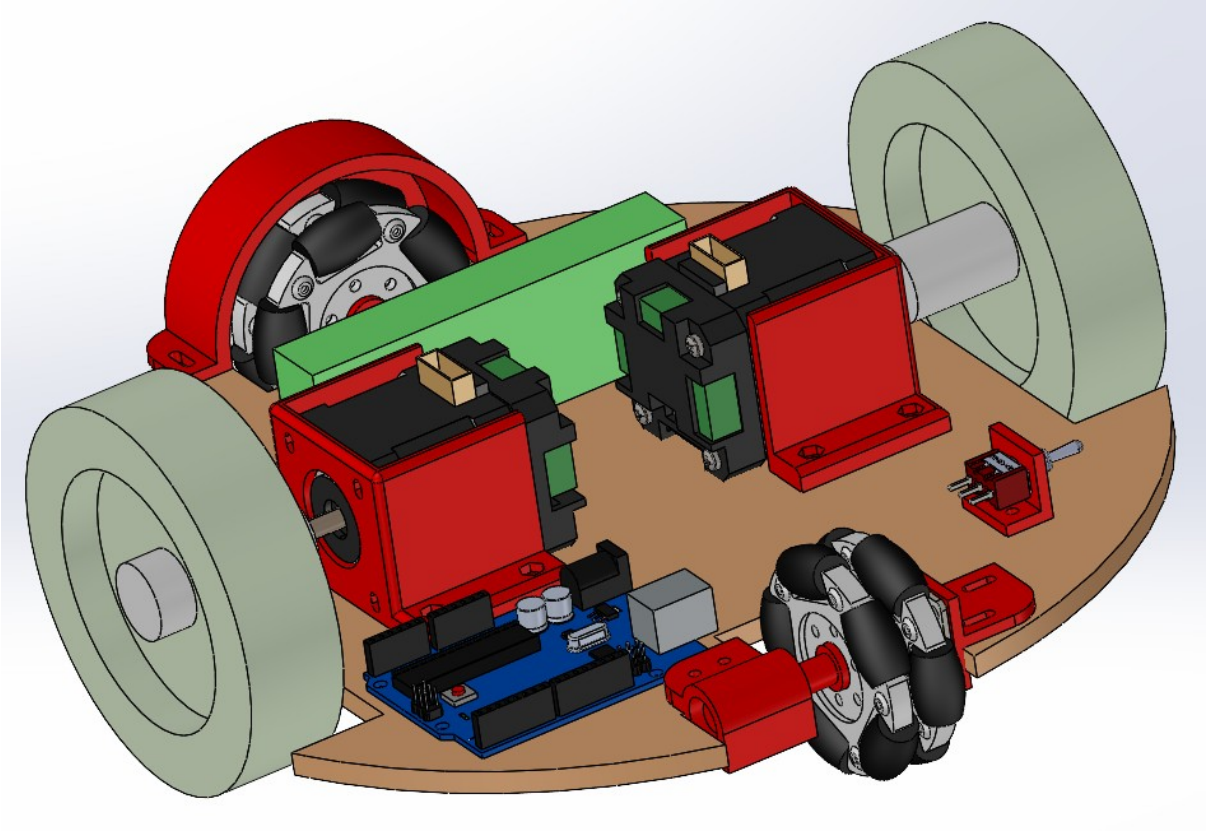


Support avant final



Support arrière final

Quand toutes les pièces ont été créées, nous avons pu instaurer des contraintes entre elles pour coller, centrer ou fixer une pièce à une distance précise d'une autre. Faire cela permet d'avoir un modèle très réaliste où chaque pièce est au bon endroit avec les bons degrés de liberté. Après avoir stylisé les pièces avec des couleurs (rouge pour les pièces que nous avons modélisées), voici le résultat final :



5.3 Partie de Kelian Raoux

Application :

Frontend :

Notre objectif était de créer une application qui nous permettrait de nous connecter en bluetooth à l'ESP32 pour lui envoyer des instructions qu'elle pourrait communiquer aux moteurs. Le problème en utilisant MIT App Inventor, c'est notre capacité à rendre l'application agréable visuellement. En effet, au début, elle ressemblait à cela :

Afin d'améliorer le rendu de l'application, nous avons appris à manier le logiciel en ajoutant des composants, en modifiant leur emplacement et en

les stylisant. Après cela nous avons pu mettre en place les différentes fonctionnalités de l’application :

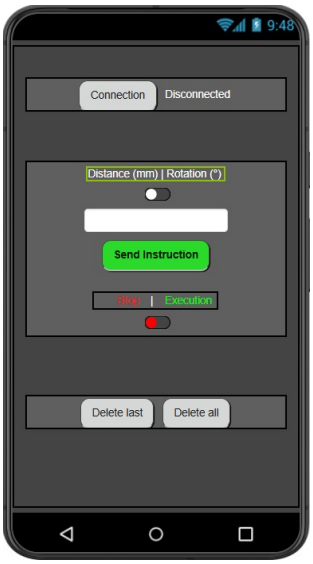
- Un switch pour que la valeur rentrée de la boîte de texte soit interprétée par le programme comme un angle pour une rotation ou comme une distance pour un déplacement.
- Un autre switch pour permettre au programme de faire défiler les instructions à venir ou l'arrêt total des moteurs.
- Des zones de textes pour que les 2 switchs soient bien différenciés l’un de l’autre.
- Deux nouveaux boutons, le premier envoyant un signal au programme pour retirer la dernière instruction stockée dans la file d’instructions, et l’autre envoyant un signal au programme pour vider entièrement la file.

Ainsi que certaines modifications visuelles pour l’application :

- Changement de la couleur du bouton d’envoi d’instruction en vert.
- Regroupement des 2 boutons pour gérer la liste d’instructions du programme pour les aligner horizontalement.
- Regroupement du bouton pour choisir à quel appareil bluetooth se connecter et du texte signifiant si l’appareil bluetooth est connecté pour les aligner horizontalement.
- Regroupement des switchs, des textes les décrivant, de la boîte de texte, et du bouton pour envoyer les instructions pour les aligner horizontalement.
- Trois espaces de texte vide permettant de créer des espaces entre les différents éléments et donc d’aérer l’application, la rendant plus agréable visuellement.

Ensuite, nous avons voulu donner un aspect plus professionnel à notre application en modifiant certains éléments :

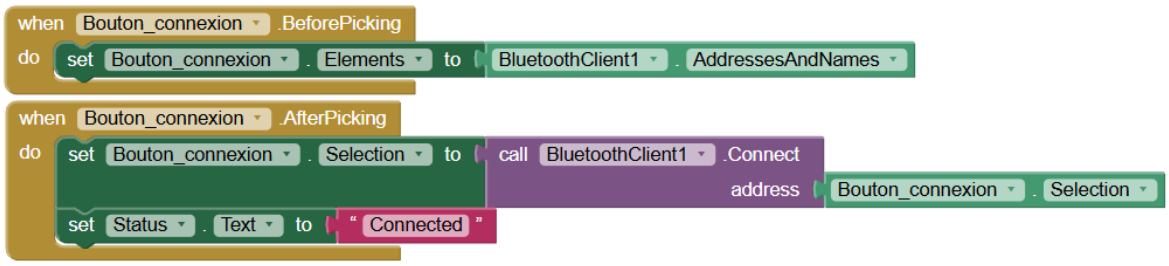
- Fond d’écran de couleur gris foncé.
- Texte en blanc pour être lisible sur fond sombre.
- Couleur des regroupements gris plus clair (#5F5F5F).
- Séparation du texte “Stop | Execution” pour que “Stop” soit écrit en rouge, le “|” en blanc et “Execution” en vert.
- Couleur du switch en rouge quand il est en OFF et vert quand il est en ON, pour correspondre avec le texte écrit juste au dessus.
- Forme des boutons en “arrondie”.



- Suppression de la couleur verte quand le switch “Déplacement | Rotation” est sur ON.

Backend :

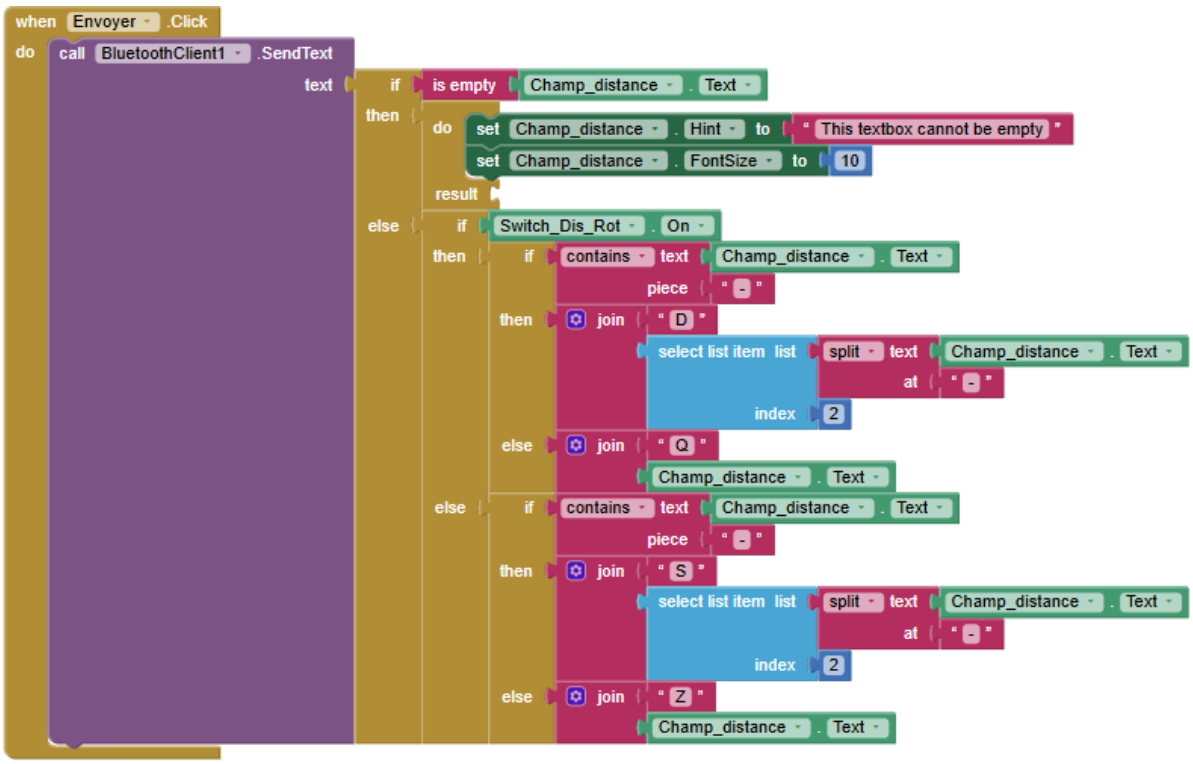
Codage des fonctions de connexion bluetooth :



La première de ces deux fonctions permet de montrer à l’écran une liste d’appareils bluetooth auxquels le téléphone peut se connecter quand on appuie sur le bouton de connexion.

La deuxième permet de se connecter à cet appareil bluetooth quand on a fait notre choix, et de changer le texte apparaissant à côté du bouton de connexion sur l’application, passant de “Disconnected” à “Connected”.

Codage du bouton d’envoi :



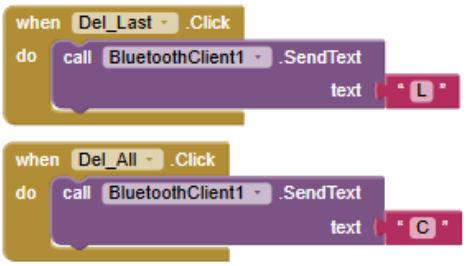
Cette fonction vérifie d’abord que la boîte de texte n’est pas vide, sinon elle écrit un message d’erreur sur la boîte vide en disant qu’elle ne peut pas être vide, et elle baisse la taille de police pour que le message rentre en entier dans la boîte de texte. Ensuite, selon si le switch “Distance | Rotation” est activé, la fonction va envoyer des informations différentes pour que le programme puisse différencier l’un de l’autre, et enfin si la valeur entrée est négative, la fonction va séparer le

message en 2 parties et n’envoyer que les nombres, accompagnées par une lettre, suivant un code qu’on a défini à l’avance que voici :

- M : exécute les instructions
- E : emergency stop
- Z : avancer
- S : reculer
- Q : rotation à gauche (sens trigonométrique)
- D : rotation à droite (sens horaire)
- C : vider la file d’instructions
- L : supprimer le dernier élément de la file d’instructions

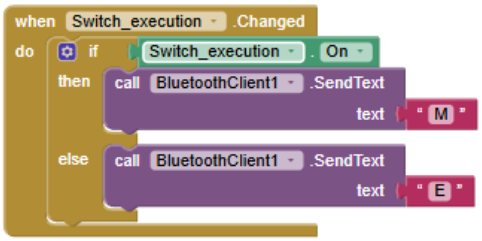
Les instructions reçues par l’ESP32 ressembleront donc par exemple à : “S140” quand on aura écrit “-140” dans la boîte de texte.

Codage des fonctions en rapport avec la file d’instructions :



La première fonction permet d’envoyer l’instruction “L” au Programme, c'est-à -dire lui demander de supprimer le dernier élément de la file d’instructions si on suit notre code, et la deuxième fonction permet d’envoyer l’instruction “C”, qui elle vide totalement la file d’instructions.

Codage de la fonction du switch “Stop | Execution” :

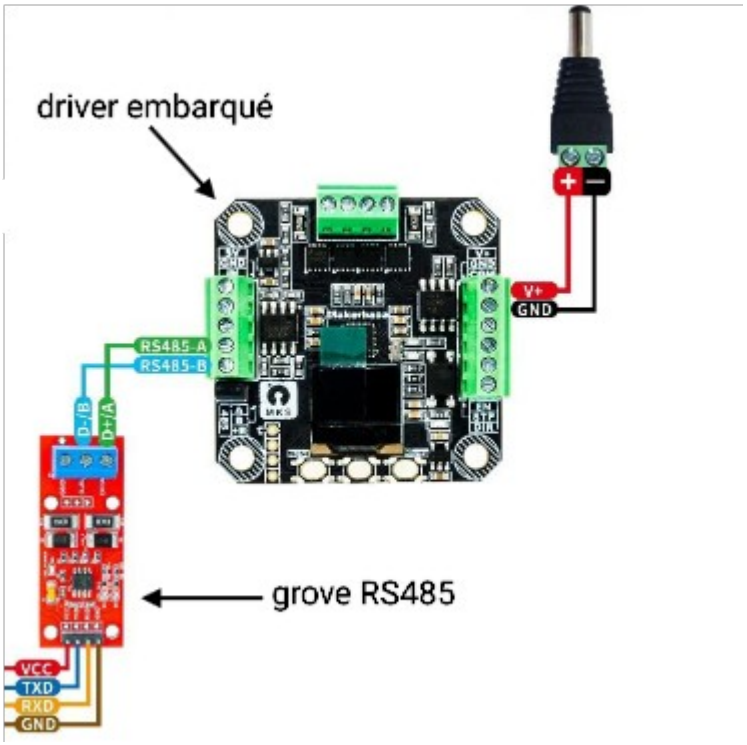


Cette fonction permet d’envoyer une instruction au programme quand le switch “Stop | Execution” change d’état. S’il passe en ON, la fonction va envoyer “M” pour dire au programme de commencer à exécuter et défiler les instructions, et s’il passe en OFF, la fonction va envoyer ‘E” pour dire au programme d’arrêter l’instruction en cours d’exécution s’il y en a une, et pour que le programme arrête de défiler la file d’instructions.

5.4 Partie de Noah Barbié Spina

Pour que le robot puisse se déplacer, nous avons utilisé des moteurs pas à pas Makerbase Servo42D avec une interface RS485 permettant la communication avec l'ESP32 par le biais du Grove RS485. Nous avons d'abord cherché à comprendre le fonctionnement des moteurs pas à pas ainsi qu'à trouver la bonne configuration pour les utiliser dans notre cas (étant plus poussés que les moteurs pas à pas classique).

Les ports que nous avons utilisés sur les moteurs sont les suivants :



- V+ (alimentation)
- GND (masse alimentation)
- RS485-A (port de communication série)
- RS485-B (port de communication série)

Configuration des moteurs :

La particularité de ces moteurs pas à pas est que la partie driver est intégrée et accompagnée de nombreux paramètres, il faut donc configurer les moteurs eux-mêmes en utilisant l'écran et les boutons intégrés.
Pour les configurer, nous avons d'abord essayé d'utiliser une vidéo explicative des fabricants (Makerbase) montrant configuration, câblage et programme mais malgré plusieurs vérifications cela n'a pas abouti.

Finalement, nos enseignants ont trouvé la solution qui se trouvait simplement dans le repo GitHub du fabricant (il y a aussi eu 'inversion des câbles RS485-A et RS485-B au niveau du shield).

Les moteurs sont donc configurés en SR_CLOSE :

SR signifie qu’on est en communication série par l’interface RS485, par opposition à CR (ou Control Reference) signifiant le contrôle par impulsion qui nous imposerait de gérer nous même de nombreuses choses comme la fréquences des pas, le timer, le profil d’accélération et celui de vitesse alors qu’avec SR, le moteur gère tout tout seul.

CLOSE signifie qu’on est en boucle fermée, c'est-à-dire que le moteur va corriger tout seul sa position, notamment grâce au PID (expliqué dans la Solution 1 de l’état de l’art).

On aurait pu utiliser le SR_vFOC mais celui-ci avait une correction (la fraction de courant de travail) trop haute (50%) qui ne pouvait pas être réduite, cela causait une oscillation continue dès que le robot faisait un mouvement car les deux moteurs essaient de corriger leur position en même temps.

En utilisant le SR_CLOSE, nous avons pu modifier le paramètre HoldMA et baisser le pourcentage à 20% ce qui a fait cesser les oscillations.

Communication avec les moteurs :

Les moteurs et l'ESP communiquent avec des commandes hexadécimales (ou trames) au format endian, elles sont composées de :

- la tête (header) (FA pour envoyer une instruction et FB pour la réponse)

- l'adresse des moteurs (slave address) (0 : broadcast, 1 : premier moteur, 2 : deuxième moteur)

- la fonction (function code) (exécute la fonction correspondante, par exemple F6 qui est le speed mode)

- les données (instruction data) (les données nécessaire à l'exécution de la fonction)

- le CRC (checksum) (permet de vérifier la fiabilité de la commande avant de l'exécuter)

downlink frames (host → SERV042/57D board)							
Frame header	Slave address	function code	Instruction data				CRC check code
FA	addr	code					CRC
Uplink frame (host← SERV042D/57D board)							
Frame header	Slave address	function code	Returned data				CRC check code
FB	addr	code					CRC

Pour voir les commandes durant la communication sans ajouter d'émetteur (la contrainte du RS485 étant qu'on ne peut avoir qu'un seul émetteur actif) nous avons plutôt intercepter les commandes en utilisant un logiciel de type serial port monitor pour "sniffer" les commandes ce qui permettait de les voir et de comprendre les quels servaient à quoi (puis nous avons simplement utilisé le logiciel "MKS servoD Control" qui nous permet de faire des tests et de voir les commandes qu'on envoyait et recevait).

Le programme :

Pour chaque fonctionnalité de notre robot (par le biais de l'application) nous avons utilisé dans le programme embarqué une fonction permettant de traduire les informations envoyés par l'application en commande (trames) compréhensible par le moteur, plusieurs de ces fonctions sont disponibles sur le github des fabricants, nous n'avons qu'a les adapter à notre cas. Voici un exemple avec la fonction "positionMode" qui va ordonner aux moteurs de tourner d'un certain nombre de pas :

```
void positionMode(uint8_t slaveAddr,uint8_t dir,uint16_t speed,uint8_t acc,uint32_t
pulses)
//speed : 1-3000 , dir : 0-1, acc : 0-255, pulse : nombre de pas
{
 //----------------------------------------
 txBuffer[0] = 0xFA;    //frame header
 txBuffer[1] = slaveAddr; //slave address
 txBuffer[2] = 0xFD;    //function code
 txBuffer[3] = (dir<<7) | ((speed>>8)&0x0F); //High 4 bits for direction and speed
 txBuffer[4] = speed&0x00FF;  //8 bits lower
 txBuffer[5] = acc;      //acceleration
 txBuffer[6] = (pulses >> 24)&0xFF; //Pulse bit31 - bit24
 txBuffer[7] = (pulses >> 16)&0xFF; //Pulse bit23 - bit16
 txBuffer[8] = (pulses >> 8)&0xFF;  //Pulse bit15 - bit8
 txBuffer[9] = (pulses >> 0)&0xFF;  //Pulse bit7 - bit0
 txBuffer[10] = getCheckSum(txBuffer,10); //Calculate checksum
 //----------------------------------------

 SerialBT.print("TX : ");

 for(int i=0;i<11;i++)
```

```
 {
  SerialBT.printf("%02X ", txBuffer[i]);
 }

 SerialBT.println();

 Serial2.write(txBuffer,11);
}
```

Récupération des données envoyés par l’application :

Comme expliqué dans la partie application mobile de Kelian Raoux, l’esp va reçevoir une valeur précédée d’une lettre qui va déterminer l’action à effectuer : le header.

On va donc récupérer ce header à chaque tour de boucle en le séparant du reste du code puis on va l'assigner à une action particulière.

Pour cela on utilisera dans la boucle principale autant de déclarations conditionnelles qu’on a de header possibles, plus une qui servira dans le cas où nous ne recevons rien.

Capteurs Hall :

Pour utiliser les encodeurs magnétiques avec capteurs d’angle à effet hall, nous allons réadapter le programme suivant qu’on à récupéré depuis internet et coupé, qui a été testé et qui nous permet d’obtenir l’angle de rotation du robot et le nombre de tours effectués, on va donc l’utiliser pour la roue libre arrière qui gère la rotation :

```
#include <Wire.h>

int magnetStatus = 0;
int lowbyte;
word highbyte;
int rawAngle; //final raw angle
float degAngle; //raw angle in degrees (360/4096 * [value between 0-4095])
float correctedAngle = 0; //tared angle
float startAngle = 0; //starting angle
float previousCorrectedAngle = 0;

void setup()
```

```
{
 Serial.begin(115200);
 Wire.begin();
 Wire.setClock(800000L);
 Serial.print("googaga");
 //checkMagnetPresence();
 ReadRawAngle();
 startAngle = degAngle;

}

void loop()
{
  ReadRawAngle(); //lit la valeur
  correctAngle(); //tare de la valeur
  previousCorrectedAngle = correctedAngle; //placer à la fin de la loop !!!!!
}

void ReadRawAngle()
{
 Wire.beginTransmission(0x36);
 Wire.write(0x0D);
 Wire.endTransmission();
 Wire.requestFrom(0x36, 1);
 while(Wire.available() == 0);
 lowbyte = Wire.read();
 Wire.beginTransmission(0x36);
 Wire.write(0x0C);
 Wire.endTransmission();
 Wire.requestFrom(0x36, 1);
 while(Wire.available() == 0);
 highbyte = Wire.read();
 highbyte = highbyte << 8;
 rawAngle = highbyte | lowbyte;
 degAngle = rawAngle * 0.087890625;
```

```
 //Serial.print("Deg angle: ");
 //Serial.println(degAngle, 2);

}

void correctAngle()
{
 correctedAngle = degAngle - startAngle; //tare de la position
 if(correctedAngle < 0)
 {
 correctedAngle = correctedAngle + 360; //normalisation de l'angle si il est
négatif
 }
 else
 {
  //do nothing
 }
 Serial.print("Corrected angle: ");
 Serial.println(correctedAngle, 2);
}

void checkMagnetPresence()
{
 while((magnetStatus & 32) != 32) //test si l'aiment est a une distance correcte
(sinon il y a des risques d'erreurs)
 {
  magnetStatus = 0;
  Wire.beginTransmission(0x36);
  Wire.write(0x0B);
  Wire.endTransmission();
  Wire.requestFrom(0x36, 1);
  while(Wire.available() == 0);
  magnetStatus = Wire.read();
 }
}
```

On l'adaptera (simple calcul avec périmètre de la roue et nombre de rotations) ensuite aussi pour avoir la distance parcourue (roue libre avant).

L’implémentation de ces capteurs n’a pas encore été faite à l’heure de l’écriture de ce rapport.

5.5 Intégration

Le principal problème que nous avons rencontré durant ce projet est le fonctionnement des moteurs pas à pas qui nous ont été donnés. Nous avons mal lu la documentation, donc nous n’avions pas les bons réglages, ce qui empêchait les moteurs de fonctionner.

Nous avons également rencontré beaucoup de problèmes en rapport avec la modélisation 3D. Beaucoup des pièces que nous avons imprimées en 3D n’avaient au final pas les bonnes dimensions pour faire ce dont nous avions besoin. Au total nous avons fait 3 sessions d’impressions. Voici quelques restes d’anciennes pièces défectueuse ou avec des mauvaises mesures ainsi que leurs support d’impression :

Nous avons aussi changé de méthode de transmission d’information, passant de wifi à bluetooth, puisque le wifi de l’université était tellement saturé quand nous avons fait nos tests que nous avons conclu que si le réseau était autant saturé le jour de l’évaluation, nous ne pourrions pas présenter notre projet dans de bonnes conditions, et que le bluetooth serait donc une solution plus stable.

De plus, nous avons rencontré un problème avec les axes que nous utilisons pour tenir les roues libres, qui au début laissaient trop de jeu sur la roue.

Nous n’avions pas non plus pensé au fait que les pins du capteur à effet Hall allaient dépasser autant, ce qui a causé un problème pour le montage du robot.

Nous avons également cassé un des axes pour tenir les roues libres, qui était creux à l’intérieur et donc pas assez solide.

Nous avons finalement dû redécouper le châssis pour s’adapter aux changements faits aux pièces imprimées en 3D. En effet, les supports pour les axes étaient au début trop courts, nous les avons donc allongés, et pour s’adapter à ce changement de taille pour ces pièces, nous avons dû légèrement redécouper le châssis.

6 Conclusion

Comme expliqué précédemment, au moment où nous rédigeons ce rapport, certaines des fonctionnalités de détection de déplacement restent à faire, ainsi que la partie correspondant au protocole R2T2. Malgré cela, nous avons pu avoir un regard objectif sur le projet et les pistes d’améliorations à lui apporter pour permettre d’obtenir de meilleurs résultats. En effet, nous aurions pu ajouter des congés à certaines des pièces pour modifier la hauteur des roues libres pour être sûr qu’elles touchent bien le sol. Ou alors, mettre des suspensions pour les plaquer au sol. Faire des rigoles pour loger les câbles de branchement afin d’avoir un résultat plus propre et moins

dangereux. Aussi, les broches du capteur hall arrière ne sont pas protégées dans le cas où le robot percute un obstacle en reculant. Il faudrait donc modéliser un cache ou alors changer le capteur de place. Mais aussi, nous pourrions modéliser un cache pour englober le robot afin de le rendre plus esthétique. De la même manière, l’application peut aussi être améliorée sur le plan esthétique.

Ce projet nous aura initié à la programmation embarquée de manière intéressante et amusante, tout en aillant l’aide de nos enseignants dans les moments difficiles pour réussir à donner un résultat satisfaisant, autant pour eux que pour nous.

Nous avons pu apprendre comment s'organise le montage d’un robot similaire au nôtre. Mais aussi constater la quantité de travail qu'il faut fournir afin d’arriver à un résultat concluant, notamment sur la partie de la modélisation 3D, où quelques millimètres seulement peuvent rendre la pièce inutilisable.

Mais surtout, la manière dont se déroule un projet du début jusqu’à la fin

Finalement, nous sommes fiers du résultat que nous avons délivré, même s’il admet encore quelques défauts.